

Les servlets

1. Présentation des servlets

Une servlet est un programme Java qui s'exécute côté serveur en tant qu'extension du serveur. Elle reçoit une requête du client, elle effectue des traitements et renvoie le résultat. La liaison entre la servlet et le client peut être directe ou passer par un intermédiaire comme par exemple un serveur http. Sun fournit des informations sur les servlets sur son site : <http://java.sun.com/products/servlet/index.html>

Même si pour le moment la principale utilisation des servlets est la génération de pages html dynamiques utilisant le protocole http et donc un serveur web, n'importe quel protocole reposant sur le principe de requête/réponse peut faire usage d'une servlet.

Ecrit en java, une servlet en retire ses avantages : la portabilité, l'accès à toutes les API de java dont JDBC pour l'accès aux bases de données, aux java beans, le garbage collector ...

L'un des principaux atouts des servlets est la réutilisabilité (réutilisation), permettant de créer des composants encapsulant des services similaires, afin de pouvoir les réutiliser dans des applications futures.

Une servlet peut être invoquée plusieurs fois en même temps pour répondre à plusieurs requêtes simultanées.

La servlet se positionne dans une architecture Client/Serveur trois tiers dans le tiers du milieu entre le client léger chargé de l'affichage et la source de données.

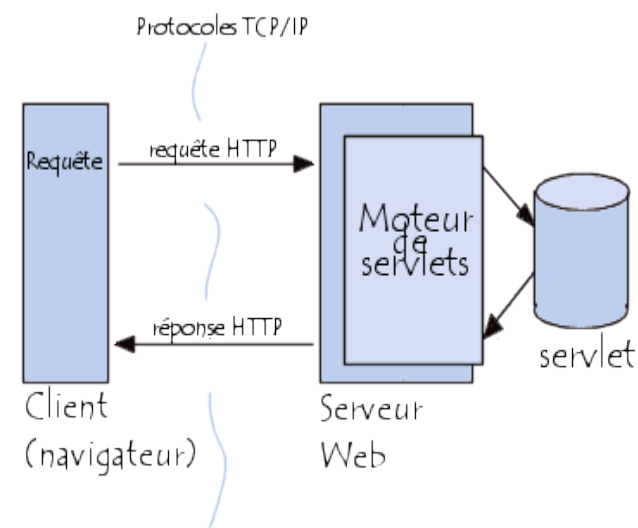
1.1. Le fonctionnement d'une servlet (cas d'utilisation de http)

Un serveur d'application permet de charger et d'exécuter les servlets dans une JVM. C'est une extension du serveur web. Ce serveur d'application contient entre autre un moteur de servlets qui se charge de manager les servlets qu'il contient.

Pour exécuter une servlet, il suffit de saisir une URL qui désigne la servlet dans un navigateur.

1. Le serveur reçoit la requête http qui nécessite une servlet de la part du navigateur
2. Si c'est la première sollicitation de la servlet, le serveur l'instancie. Les servlets sont stockées (sous forme de fichier .class) dans un répertoire particulier du serveur. Ce répertoire dépend du serveur d'application utilisé. La servlet reste en mémoire jusqu'à l'arrêt du serveur ce qui permet de garder des ressources systèmes et gagner le temps de l'initialisation par rapport aux CGI et les langages de script côté serveur (PHP, ASP) qui sont chargés en mémoire à chaque requête, ce qui réduit les performances.. Certains serveurs d'application permettent aussi d'instancier des servlets dès le lancement du serveur.

La servlet en mémoire, peut être appelée par plusieurs threads lancés par le serveur pour chaque requête. Ce principe de fonctionnement évite d'instancier un objet de type servlet à chaque requête et permet de maintenir un ensemble de ressources actives tel qu'une connexion à une base de données.
3. le serveur crée un objet qui représente la requête http et objet qui contiendra la réponse et les envoie à la servlet
4. la servlet crée dynamiquement la réponse sous forme de page html transmise via un flux dans l'objet contenant la réponse. La création de cette réponse utilise bien sûr la requête du client mais aussi un ensemble de ressources incluses sur le serveur tels que des fichiers ou des bases de données.
5. le serveur récupère l'objet réponse et envoie la page html au client.



2. L'API servlet

Les servlets sont conçues pour agir selon un modèle de requête/reponse. Tous les protocoles utilisant ce modèle peuvent être utilisés tel que http, ftp, etc ...

L'API servlets est une extension du jdk de base, et en tant que telle elle est regroupée dans des packages préfixés par javax

L'API servlet regroupe un ensemble de classes dans deux packages :

- javax.servlet : contient les classes pour développer des servlets génériques indépendantes d'un protocole
- javax.servlet.http : contient les classes pour développer des servlets qui reposent sur le protocole http utilisé par les serveurs web.

Le package javax.servlet définit plusieurs interfaces, méthodes et exceptions :

javax.servl et	Nom	Role
Les interfaces	RequestDispatcher	Définition d'un objet qui permet le renvoi d'une requête vers une autre ressource du serveur (une autre servlet, une JSP ...)

	Servlet	Définition de base d'une servlet
	ServletConfig	Définition d'un objet pour configurer la servlet
	ServletContext	Définition d'un objet pour obtenir des informations sur le contexte d'exécution de la servlet
	ServletRequest	Définition d'un objet contenant la requête du client
	ServletResponse	Définition d'un objet qui contient la réponse renvoyée par la servlet
	SingleThreadModel	Permet de définir une servlet qui ne répondra qu'à une seule requête à la fois
Les classes	GenericServlet	Classe définissant une servlet indépendante de tout protocole
	ServletInputStream	Flux permet la lecture des données de la requête cliente
	ServletOutputStream	Flux permettant l'envoi de la réponse de la servlet
Les exceptions	ServletException	Exception générale en cas de problème de la servlet
	UnavailableException	Exception levée si la servlet n'est pas disponible

Le package javax.servlet.http définit plusieurs interfaces et méthodes :

Javax.servl et	Nom	Role
Les interfaces	HttpServletRequest	Hérite de ServletRequest : définit un objet contenant une requête selon le protocole http
	HttpServletResponse	Hérite de ServletResponse : définit un objet contenant la réponse de la servlet selon le protocole http

	HttpSession	Définit un objet qui représente une session
Les classes	Cookie	Classe représentant un cookie (ensemble de données sauvegardées par le browser sur le poste client)
	HttpServlet	Hérite de GenericServlet : classe définissant une servlet utilisant le protocole http
	HttpUtils	Classe proposant des méthodes statiques utiles pour le développement de servlet http

2.1. L'interface Servlet

Une servlet est une classe Java qui implémente l'interface `javax.servlet.Servlet`. Cette interface définit 5 méthodes qui permettent au serveur de dialoguer avec la servlet : elle encapsule ainsi les méthodes nécessaires à la communication entre le serveur et la servlet.

Méthode	Rôle
<code>void service (ServletRequest req, ServletResponse res)</code>	Cette méthode est exécutée par le serveur lorsque la servlet est sollicitée : chaque requête du client déclenche une seule exécution de cette méthode. Cette méthode pouvant être exécutée par plusieurs threads, il faut prévoir un processus d'exclusion pour l'utilisation de certaines ressources.
<code>void init(ServletConfig conf)</code>	Initialisation de la servlet. Cette méthode est appelée une seule fois après l'instanciation de la servlet. Aucun traitement ne peut être effectué par la servlet tant que l'exécution de cette méthode n'est pas terminée.

<code>ServletConfig getServletConfig()</code>	Renvoie l'objet <code>ServletConfig</code> passé à la méthode <code>init</code>
<code>void destroy()</code>	Cette méthode est appelée lors la destruction de la servlet. Elle permet de libérer proprement certaines ressources (fichiers, bases de données ...). C'est le serveur qui appelle cette méthode.
<code>String getServletInfo()</code>	Renvoie des informations sur la servlet.

Les méthodes **`init()`**, **`service()`** et **`destroy()`** assure le **cycle de vie** de la servlet en étant respectivement appelée lors de la création de la servlet, lors de son appel pour le traitement d'une requête et lors de sa destruction.

La méthode `init()` est appelée par le serveur juste après l'instanciation de la servlet.

La méthode `service()` ne peut pas être invoquée tant que la méthode `init` n'est pas terminée.

La méthode `destroy()` est appelée juste avant que le serveur ne détruise la servlet : cela permet de libérer des ressources allouées dans la méthode `init()` tel qu'un fichier ou une connexion à une base de données.

2.3. La requête et la réponse

L'interface `ServletRequest` définit plusieurs méthodes qui permettent d'obtenir les données sur la requête du client:

Méthode	Role
<code>ServletInputStream getInputStream()</code>	Permet d'obtenir un flux pour les données de la requête
<code>BufferedReader getReader()</code>	Idem

L'interface `ServletResponse` définit plusieurs méthodes qui permettent de fournir la réponse faite par la servlet suite à ces traitements :

Méthode	Role
<code>SetContentType</code>	Permet de préciser le type MIME de la réponse
<code>ServletOutputStream getOutputStream()</code>	Permet d'obtenir un flux pour envoyer la réponse

PrintWriter getWriter()	Permet d'obtenir un flux pour envoyer la réponse
-------------------------	--

2.2 . Un exemple de servlet

Une servlet qui implémente simplement l'interface Servlet doit évidemment redéfinir toute les méthodes définies dans l'interface.

Il est très utile lorsque que l'on crée une servlet qui implémente directement l'interface Servlet de sauvegarder l'objet ServletConfig fourni par le serveur en paramètre de la méthode init() car c'est le seul moment ou l'on a accès à cet objet.

Exemple (code java 1.1) :

```
import java.io.*;
import javax.servlet.*;

public class TestServlet implements Servlet {
    private ServletConfig cfg;

    public void init(ServletConfig config) throws ServletException {
        cfg = config;
    }

    public ServletConfig getServletConfig() {
        return cfg;
    }

    public String getServletInfo() {
        return "Une servlet de test";
    }

    public void destroy() {
    }

    public void service (ServletRequest req, ServletResponse res )
        throws ServletException, IOException {
```

```
        res.setContentType( "text/html" );
        // Recherche du flux d'écriture
        PrintWriter out = res.getWriter();
        // Ecriture de la page reponse
        out.println( "<THML>" );
        out.println( "<HEAD>" );
        out.println( "<TITLE>Page generee par une servlet</TITLE>" );
        out.println( "</HEAD>" );
        out.println( "<BODY>" );
        out.println( "<H1>Bonjour</H1>" );
        out.println( "</BODY>" ); out.println( "</HTML>" );
        out.close();
    }
}
```

3. Le protocole HTTP

Le protocole HTTP est un protocole qui fonctionne sur le modèle client/serveur. Un client qui est une application (souvent un navigateur web) envoie une requête à un serveur (un serveur web). Ce serveur attend en permanence les requêtes sur un port particulier (par défaut le port 80). A la réception de la requête, le serveur lance un thread qui va la traiter pour générer la réponse. Le serveur renvoie la réponse au client une fois les traitements terminés.

Une particularité du protocole HTTP est de maintenir la connexion entre le client et le serveur uniquement durant l'échange de la requête et de la réponse.

Il existe deux versions principales du protocole HTTP : 1.0 et 1.1.

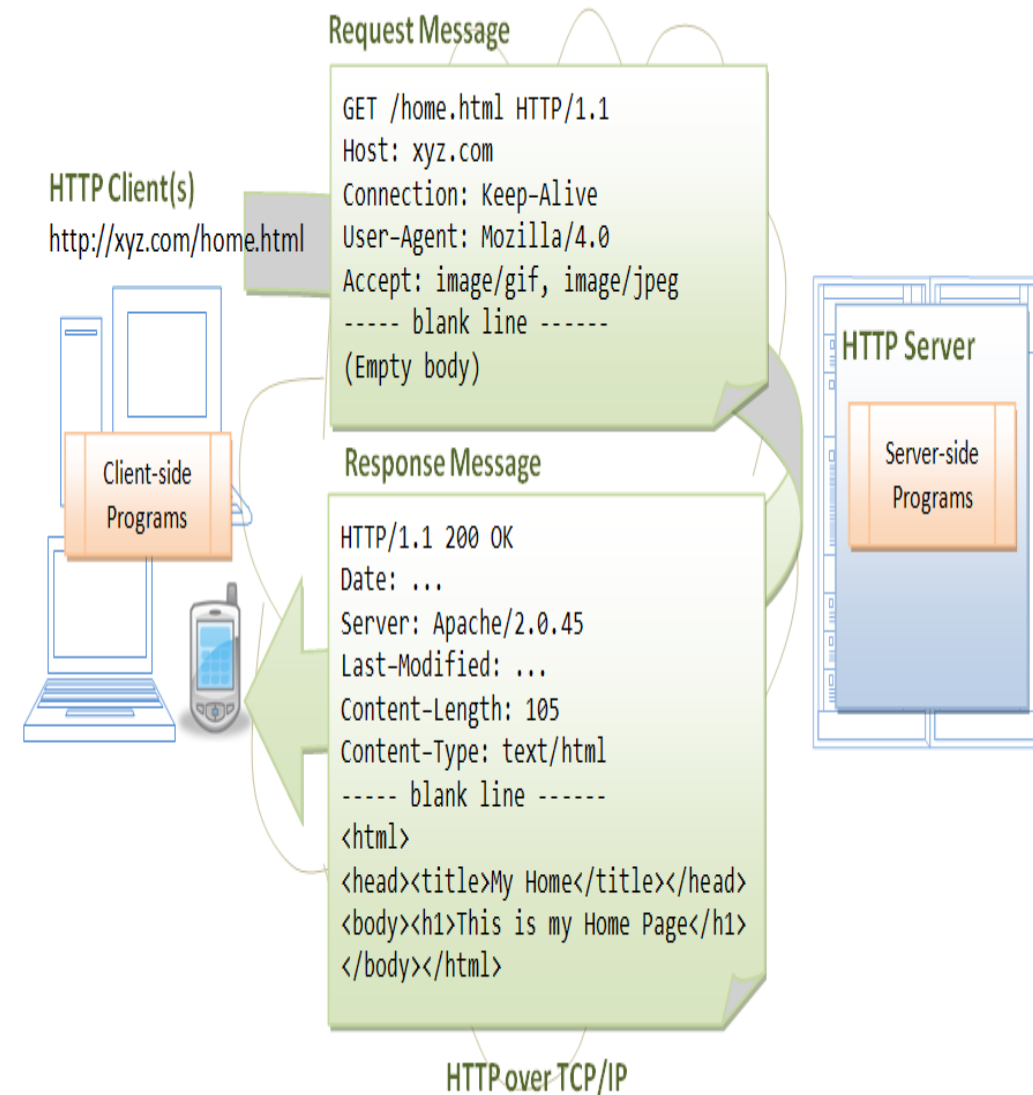
La requête est composée de trois parties :

- la commande
- la section en-tête
- le corps

La première ligne de la requête contient la commande à exécuter par le serveur. La commande est suivie éventuellement d'un argument qui précise la commande (par exemple l'url de la ressource demandée). Enfin la ligne doit contenir la version du protocole HTTP utilisé, précédée de HTTP/.

Exemple :

Avec HTTP 1.1, les commandes suivantes sont définies : GET, POST, HEAD, OPTIONS, PUT, DELETE, TRACE et CONNECT. Les trois premières sont les plus utilisées.



Il est possible de fournir sur les lignes suivantes de la partie en-tête des paramètres supplémentaires. Cette partie en-tête est optionnelle. Les informations fournies peuvent permettre au serveur d'obtenir des

informations sur le client. Chaque information doit être mise sur une ligne unique. Le format est `nom_du_champ:valeur`. Les champs sont prédéfinis et sont sensibles à la casse.

Une ligne vide doit précéder le corps de la requête. Le contenu du corps de la requête dépend du type de la commande.

La requête doit obligatoirement être terminée par une ligne vide.

La réponse est elle aussi composée des trois mêmes parties :

- une ligne de statuts
- un en-tête dont le contenu est normalisé
- un corps dont le contenu dépend totalement de la requête

La première ligne de l'en-tête contient un état qui est composé : de la version du protocole HTTP utilisé, du code de statut et d'une description succincte de ce code.

Le code de statut est composé de trois chiffres qui donnent des informations sur le résultat du traitement qui a généré cette réponse. Ce code peut être regroupé en plusieurs catégories en fonction de leur valeur :

Plage de valeur du code	Signification
100 à 199	Information
200 à 299	Traitement avec succès
300 à 399	La requête a été redirigée
400 à 499	La requête est incomplète ou erronée
500 à 599	Une erreur est intervenue sur le serveur

Plusieurs codes sont définis par le protocole HTTP dont les plus importants sont :

- 200 : traitement correct de la requête
- 204 : traitement correct de la requête mais la réponse ne contient aucun contenu (ceci permet au browser de laisser la page courante affichée)
- 404 : la ressource demandée n'est pas trouvée (sûrement le plus célèbre)
- 500 : erreur interne du serveur

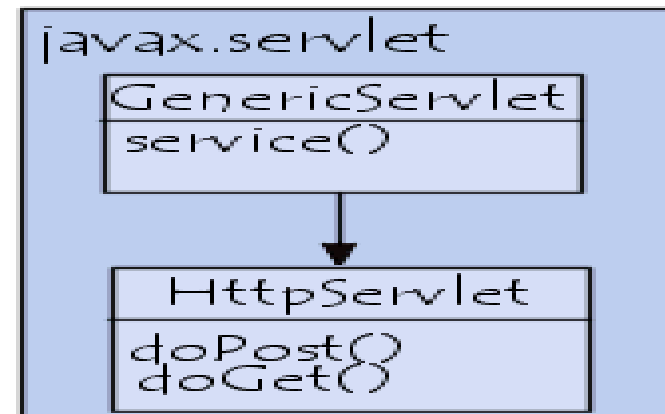
L'en-tête contient des informations qui précisent le contenu de la réponse.

Le corps de la réponse est précédé par une ligne vide.

3.1. Les servlets http

L'usage principale des servlets est la création de pages HTML dynamiques. Sun fournit une classe qui encapsule une servlet utilisant le protocole http. Cette classe est la classe `HttpServlet`

Cette classe hérite de `GenericServlet`, donc elle implémente l'interface `Servlet`, et redéfinit toutes les méthodes nécessaires pour fournir un niveau d'abstraction permettant de développer facilement des servlets avec le protocole http.



Ce type de servlet n'est pas utile que pour générer des pages HTML bien que cela soit son principal usage, elle peut aussi réaliser un ensemble de traitements tel que mettre à jour une base de données. En réponse, elle peut générer une page html qui indique le succès ou non de la mise à jour.

Elle définit un ensemble de fonctionnalités très utiles : par exemple, elle contient une méthode **`service()`** qui appelle certaines méthodes à redéfinir en fonction du type de requête http (`doGet()`, `doPost()`, etc ...).

La requête du client est encapsulée dans un objet qui implémente l'interface `HttpServletRequest` : cet objet contient les données de la requête et des informations sur le client.

La réponse de la servlet est encapsulée dans un objet qui implémente l'interface `HttpServletResponse`.

Typiquement pour définir une servlet, il faut définir une classe qui hérite de la classe `HttpServlet` et redéfinir la classe `doGet` et/ou `doPost` selon les besoins.

La méthode `service` héritée de `HttpServlet` appelle l'une ou l'autre de ces méthodes en fonction du type de la requête http :

- une requête GET : c'est une requête qui permet au client de demander une page html
- une requête POST : c'est une requête qui permet au client d'envoyer des informations issues par exemple d'un formulaire

Une servlet peut traiter un ou plusieurs types de requêtes grâce à plusieurs autres méthodes :

La classe `HttpServlet` hérite aussi de plusieurs méthodes définies dans l'interface `Servlet` : `init`, `destroy` et `getServletInfo`.

3.2. La méthode `init()`

Si cette méthode doit être redéfinie, il est important d'invoquer la méthode héritée avec un `super.init(config)`, `config` étant l'objet fourni en paramètre de la méthode. La méthode définie dans la classe `HttpServlet` sauvegarde l'objet de type `ServletConfig`.

De plus, la classe `GenericServlet` implémente l'interface `ServletConfig`. Les méthodes redéfinies pour cette interface utilisent l'objet sauvegardé. Ainsi, la servlet peut utiliser sa propre méthode `getInitParameter()` ou utiliser la méthode `getInitParameter()` de l'objet de type `ServletConfig`. La première solution permet un usage plus facile dans toute la servlet.

Sans l'appel à la méthode héritée lors d'une redéfinition, la méthode `getInitParameter()` de la servlet levera une exception de type `NullPointerException`.

3.2. L'analyse de la requête

La méthode `service()` est la méthode qui est appelée lors d'un appel à la servlet.

Par défaut dans la classe `HttpServlet`, cette méthode contient du code qui réalise une analyse de la requête client contenue dans l'objet `HttpServletRequest`. Selon le type de requête GET ou POST, elle appelle la méthode `doGet()` ou `doPost()`. C'est bien ce type de requête qui indique quelle méthode utiliser dans la servlet.

Ainsi, la méthode `service()` n'est pas à redéfinir pour ces requêtes et il suffit de redéfinir les méthodes `doGet()` et/ou `doPost()` selon les besoins.

3.3. La méthode `doGet()`

Une requête de type GET est utile avec des liens d'URL du type :
`http://nom_du_serveur/repertoire/document?champ1=valeur1&champ2=valeur2...`

Toutefois, la longueur de la chaîne URL étant limitée à 255 caractères, les informations situées au-delà de cette limite seront irrémédiablement perdues.

. Par exemple :

```
<A
HREF="http://localhost:8080/examples/servlet/tomcat1.MyHelloServlet">test
de la servlet</A>
```

Dans une servlet de type `HttpServlet`, une telle requête est associée à la méthode `doGet`.

La signature de la méthode `doGet()` :

```
protected void doGet(HttpServletRequest req,
HttpServletResponse resp) throws IOException
{
}
```

Le traitement typique de la méthode `doGet()` est d'analyser les paramètres de la requête, alimenter les données de l'en-tête de la réponse et d'écrire la réponse.

3.4. La méthode doPost()

Un requête POST n'est utilisable qu'avec un formulaire HTML.

Cette méthode code les informations de la même façon que la méthode GET (encodage URL et paires nom/valeur) mais elle envoie les données à la suite des en-têtes HTTP, dans un champ appelé *corps de la requête*. De cette façon la quantité de données envoyées n'est plus limitée, et est connue du serveur grâce à l'en-tête permettant de connaître la taille du *corps de la requête*.

Exemple de code HTML :

```
<FORM
ACTION="http://localhost:8080/examples/servlet/tomcat1.TestPost
Servlet"
METHOD="POST">
<INPUT NAME="NOM">
<INPUT NAME="PRENOM">
<INPUT TYPE="ENVOYER">
</FORM>
```

Dans l'exemple ci dessus, le formulaire comporte deux zones de saisies correspondant à deux paramètres : NOM et PRENOM.

Dans une servlet de type HttpServlet, une telle requête est associée à la méthode doPost().

La signature de la méthode doPost() :

```
protected void doPost(HttpServletRequest request,
HttpServletRequest response) throws IOException
{
```

```
}
```

La méthode doPost() doit généralement recueillir les paramètres pour les traiter et générer la réponse. Pour obtenir la valeur associée à chaque paramètre il faut utiliser la méthode **getParameter** de l'objet HttpServletRequest. Cette méthode attend en paramètre le nom du paramètre dont on veut la valeur. Ce paramètre est sensible à la casse.

Exemple :

```
public void doPost(HttpServletRequest request,
HttpServletRequest response) throws IOException,
ServletException
{
    String nom = request.getParameter("NOM");
    String prenom = request.getParameter("PRENOM");
}
```

3.5. La génération de la réponse

La servlet envoie sa réponse au client en utilisant un objet de type HttpServletResponse. HttpServletResponse est une interface : il n'est pas possible d'instancier un tel objet mais le moteur de servlet instancie un objet qui implémente cette interface et le passe en paramètre de la méthode service.

Cette interface possède plusieurs méthodes pour mettre à jour l'en-tête http et le page HTML de retour.

Méthode	Rôle
void sendError (int)	Envoie une erreur avec un code retour et un message par défaut
void sendError (int, String)	Envoie une erreur avec un code retour et un message
void setContentType(String)	Héritée de ServletResponse, cette méthode permet de préciser le type MIME de la réponse

<code>void setContentLength(int)</code>	Héritée de <code>ServletResponse</code> , cette méthode permet de préciser la longueur de la réponse
<code>ServletOutputStream getOutputStream()</code>	Héritée de <code>ServletResponse</code> , elle retourne un flux pour l'envoi de la réponse
<code>PrintWriter getWriter()</code>	Héritée de <code>ServletResponse</code> , elle retourne un flux pour l'envoi de la réponse

Avant de générer la réponse sous forme de page HTML, il faut indiquer dans l'en tête du message http, le type mime du contenu du message. Ce type sera souvent « text/html » qui correspond à une page HTML mais il peut aussi prendre d'autre valeur en fonction de ce que retourne la servlet (une image par exemple). La méthode à utiliser est `setContentType`.

Il est aussi possible de préciser la longueur de la réponse avec la méthode `setContentLength()`. Cette précision est optionnelle mais si elle est utilisée, la longueur doit être exacte pour éviter des problèmes.

Il est préférable de créer une ou plusieurs méthodes recevant en paramètre l'objet `HttpServletResponse` qui seront dédiées à la génération du code HTML afin de ne pas alourdir les méthodes `doXXX()`.

Il existe plusieurs façons de générer une page HTML : elles utiliseront toutes soit la méthode `getOutputStream()` ou `getWriter()` pour obtenir un flux dans lequel la réponse sera envoyée.

- **Utilisation d'un `StringBuffer` et `getOutputStream`**

Exemple (code java 1.1) :

```
protected void GenererReponse1(HttpServletResponse
reponse) throws IOException
{
    //creation de la reponse
    StringBuffer sb = new StringBuffer();
    sb.append("<HTML>\n");
```

```
sb.append("<HEAD>\n");
sb.append("<TITLE>Bonjour</TITLE>\n");
sb.append("</HEAD>\n");
sb.append("<BODY>\n");
sb.append("<H1>Bonjour</H1>\n");
sb.append("</BODY>\n");
sb.append("</HTML>");

// envoie des infos de l'en tete
reponse.setContentType("text/html");
reponse.setContentLength(sb.length());

// envoie de la reponse
reponse.getOutputStream().print(sb.toString());
}
```

- L'avantage de cette méthode est qu'elle permet facilement de déterminer la longueur de la réponse.
- Dans l'exemple, l'ajout des retours chariot '\n' à la fin de chaque ligne n'est pas obligatoire mais elle facilite la compréhension du code HTML surtout si il devient plus complexe.
- **Utilisation directe de `getOutputStream`**

Exemple :

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class TestServlet4 extends HttpServlet {

    public void doGet(HttpServletRequest req, HttpServletResponse
res)
        throws ServletException, IOException {
        res.setContentType("text/html");
        ServletOutputStream out = res.getOutputStream();
        out.println("<HTML>\n");
        out.println("<HEAD>\n");
```

```

out.println("<TITLE>Bonjour</TITLE>\n");
out.println("</HEAD>\n");
out.println("<BODY>\n");
out.println("<H1>Bonjour</H1>\n");
out.println("</BODY>\n");
out.println("</HTML>");
}
}

```

- **Utilisation de la méthode `getWriter()`**

Exemple (code java 1.1) :

```

protected void GenererReponse2(HttpServletResponse
reponse) throws IOException {

    reponse.setContentType("text/html");
    PrintWriter out = reponse.getWriter();
    out.println("<HTML>");
    out.println("<HEAD>");
    out.println("<TITLE>Bonjour</TITLE>");
    out.println("</HEAD>");
    out.println("<BODY>");
    out.println("<H1>Bonjour</H1>");
    out.println("</BODY>");
    out.println("</HTML>");
}

```

- Avec cette méthode, il faut préciser le type MIME avant d'écrire la réponse. L'emploi de la méthode `println()` permet d'ajouter un retour chariot en fin de chaque ligne.
- Si un problème survient lors de la génération de la réponse, la méthode `sendError()` permet de renvoyer une erreur au client : un code retour est positionné dans l'en tête http et le message est indiqué dans une simple page HTML.

3.6. Un exemple de servlet HTTP très simple

Toute servlet doit au moins importer trois packages : `java.io` pour la gestion des flux et deux packages de l'API servlet ; `javax.servlet.*` et `javax.servlet.http`.

Il faut déclarer une nouvelle classe qui hérite de `HttpServlet`.

Il faut redéfinir la méthode `doGet()` pour y insérer le code qui va envoyer dans un flux le code HTML de la page générée.

Exemple (code java 1.1) :

```

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

// public class MyHelloServlet extends GenericServlet
public class MyHelloServlet extends HttpServlet {

    public void doGet(HttpServletRequest request, HttpServletResponse
response) throws IOException, ServletException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<html>");
        out.println("<head>");
        out.println("<title>Bonjour tout le monde</title>");
        out.println("</head>");
        out.println("<body>");
        out.println("<h1>Bonjour tout le monde</h1>");
        out.println("</body>");
        out.println("</html>");
        public void doPost(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException
        {
            doGet(req, res) ;
        }
    }
}

```

```
}

```

La méthode `getWriter()` de l'objet `HttpServletResponse` renvoie un flux de type `PrintWriter` dans lequel on peut écrire la réponse.

Si aucun traitement particulier n'est associé à une requête de type POST, il est pratique de demander dans la méthode `doPost()` d'exécuter la méthode `doGet()`. Dans ce cas, la servlet est capable de renvoyer une réponse pour les deux types de requête.

Exemple :

```
public void doPost(HttpServletRequest request, HttpServletResponse response)
throws ServletException, IOException {

    this.doGet(request, response);

}
```

4. Les informations sur l'environnement d'exécution des servlets

Une servlet est exécutée dans un contexte particulier mis en place par le moteur de servlet. La servlet peut obtenir des informations sur ce contexte. La servlet peut aussi obtenir des informations à partir de la requête du client.

4.1. Les paramètres d'initialisation

Dès que la servlet est instanciée, le moteur de servlet appelle sa méthode `init()` en lui donnant en paramètre un objet de type `ServletConfig`.

`ServletConfig` est une interface qui possède une méthode permettant de connaître les paramètres d'initialisation :

- `String getInitParameter(String);`

Exemple :

```
String param;
public void init(ServletConfig config) {
    param = config.getInitParameter("param");
}
```

- Enumeration `getInitParameterNames()` : retourne une énumération des paramètres d'initialisation

Exemple :

```
public void init(ServletConfig config) throws ServletException {

    cfg = config;
    System.out.println("Liste des parametres d'initialisation");

    for (Enumeration e=config.getInitParameterNames();
e.hasMoreElements();) {

        System.out.println(e.nextElement());

    }

}
```

La déclaration des paramètres d'initialisation dépend du serveur qui est utilisé.

4.2. L'objet `ServletContext`

La servlet peut obtenir des informations à partir d'un objet `ServletContext` retourné par la méthode `getServletContext()` d'un objet `ServletConfig`.

Il est important de s'assurer que cet objet ServletConfig, obtenu par la méthode init() est soit explicitement sauvegardé soit sauvegardé par l'appel à la méthode init() héritée qui effectue cette sauvegarde.

L'interface ServletContext contient plusieurs méthodes dont les principales sont :

méthode	Role	Deprecated
String getMimeType(String)	Retourne le type MIME du fichier en paramètre	
String getServletInfo()	Retourne le nom et le numero de version du moteur de servlet	
Servlet getServlet(String)	Retourne une servlet à partir de son nom grace au context	Ne plus utiliser depuis la version 2.1 du jsdk
Enumeration getServletNames()	Retourne une enumeration qui contient la liste des servlets realatives au contexte	Ne plus utiliser depuis la version 2.1 du jsdk
void log(Exception, String)	Ecrit les informations fournies en paramètre dans le fichier log du serveur	Utiliser la nouvelle méthode surchargée de log()
void log(String)	Idem	
void log (String, Throwable)	Idem	

Exemple : écriture dans le fichier log du serveur :

```
public void init(ServletConfig config) throws
ServletException {
    ServletContext sc = config.getServletContext();
    sc.log( "Demarrage servlet TestServlet" );
}
```

Le format du fichier log est dependant du serveur utilisé :

Exemple : résultat avec tomcat

Context log path="/examples" :Demarrage servlet TestServlet

4.3. Les informations contenues dans la requête :les variables d'environnement

De nombreuses informations en provenance du client peuvent être extraites de l'objetServletRequest passé en paramètre par le serveur (ou de HttpServletRequest qui hérite de ServletRequest).

Les informations les plus utiles sont les paramètres envoyés dans la requête.

L'interface ServletRequest dispose de nombreuses méthodes pour obtenir ces informations :

Méthode	Role
int getContentLength()	Renvoie la taille de la requête, 0 si elle est inconnue
String getContentType()	Renvoie le type MIME de la requête, null si il est inconnu
ServletInputStream getInputStream()	Renvoie un flux qui contient le corps de la requête
Enumeration getParameterNames()	Renvoie une énumération contenant le nom de tous les paramètres
String getProtocol()	Retourne le nom du protocole et sa version utilisé par la requête
BufferedReader getReader()	Renvoie un flux qui contient le corps de la requête
String getRemoteAddr()	Renvoie l'adresse IP du client
String getRemoteHost()	Renvoie le nom de la machine cliente
String getScheme	Renvoie le protocole utilisé par la requête (exemple : http, ftp ...)
String getServerName()	Renvoie le nom du serveur qui à reçu la

	requête
int getServerPort()	Renvoie le port du serveur qui a reçu la requête

Exemple

```
package tomcat1;
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.util.*;

public class InfoServlet extends HttpServlet {

    public void doGet(HttpServletRequest request, HttpServletResponse
response)
    throws IOException, ServletException {
        GenererReponse(request, response);
    }

    protected void GenererReponse(HttpServletRequest request,
HttpServletResponse response) throws IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<html>");
        out.println("<body>");
        out.println("<head>");
        out.println("<title>Informations a disposition de la servlet</title>");
        out.println("</head>");
        out.println("<body>");
        out.println("<p>Type mime de la requête : "
+request.getContentType()+"</p>");
        out.println("<p>Protocole de la requête : "
+request.getProtocol()+"</p>");
        out.println("<p>Adresse IP du client : "
+request.getRemoteAddr()+"</p>");
        out.println("<p>Nom du client : "
```

```
+request.getRemoteHost()+"</p>");
out.println("<p>Nom du serveur qui a reçu la requête : "
+request.getServerName()+"</p>");
out.println("<p>Port du serveur qui a reçu la requête : "
+request.getServerPort()+"</p>");
out.println("<p>scheme: "+request.getScheme()+"</p>");
out.println("<p>Liste des paramètres </p>");

for (Enumeration e = request.getParameterNames() ;
e.hasMoreElements() ; ) {
    Object p = e.nextElement();
    out.println("<p>&nbsp;&nbsp;&nbsp;nom : "+p+" valeur : "
+request.getParameter(p)+"</p>");
}
out.println("</body>");
out.println("</html>");
}
}
```

5. HTTP: un protocole non connecté

Le protocole HTTP est un protocole non connecté (on parle aussi de *protocole sans états*, en anglais *stateless protocol*), cela signifie que chaque requête est traitée indépendamment des autres et qu'aucun historique des différentes requêtes n'est conservé. Ainsi le serveur web ne peut pas se "souvenir" de la requête précédente, ce qui est dommageable dans des utilisations telles que le e-commerce, pour lequel le serveur doit mémoriser les achats de l'utilisateur sur les différentes pages.

Il s'agit donc de maintenir la cohésion entre l'utilisateur et la requête, c'est-à-dire :

- reconnaître les requêtes provenant du même utilisateur
- associer un profil à l'utilisateur
- connaître les paramètres de l'application (nombre de produits vendus, ...)

On appelle ce mécanisme de gestion des états le **suivi de session** (en anglais *session tracking*).

5. 1. L'utilisation des cookies

Les cookies sont des fichiers contenant des données au format texte, envoyés par le serveur et stockés sur le poste client. Les données contenues dans le cookie sont envoyées au serveur à chaque requête.

Les cookies peuvent être utilisés explicitement ou implicitement par exemple lors de l'utilisation d'une session.

Les cookies ne sont pas dangereux car ce sont uniquement des fichiers textes qui ne sont pas exécutés. De plus les navigateurs posent des limites sur le nombre (en principe 20 cookie pour un même serveur) et la taille des cookies (4ko maximum). Par contre les cookies peuvent contenir des données plus ou moins sensibles.

5.1.1 La classe Cookie

La classe `javax.servlet.http.Cookie` encapsule un cookie.

Un cookie est composé d'un nom, d'une valeur et d'attributs.

Pour créer un cookie, il suffit d'instancier un nouvel objet `Cookie`. La classe `Cookie` ne possède qu'un seul constructeur qui attend deux paramètres de type `String` : le nom et la valeur associée.

5.1.2 L'enregistrement et la lecture d'un cookie

Pour envoyer un cookie au browser, il suffit d'utiliser la méthode `addCookie()` de la classe `HttpServletResponse`.

Exemple :

```
// Création de la cookie
Cookie monCookie = new
Cookie("nom","valeur");
// définition de la limite de validité
monCookie.setMaxAge(24*3600);
// envoi du cookie dans la réponse
```

HTTP

```
response.addCookie(monCookie);
```

Pour lire un cookie envoyé par le browser, il faut utiliser la méthode `getCookies()` de la classe `HttpServletRequest`. Cette méthode renvoie un tableau d'objet `Cookie`. Les cookies sont renvoyés dans l'en-tête de la requête http. Pour rechercher un cookie particulier, il faut parcourir le tableau et rechercher le cookie à partir de son nom grâce à la méthode `getName()` de l'objet `Cookie`.

Exemple :

```
Cookie[] cookies = request.getCookies();
String valeur = "";
for(int i=0;i<cookies.length;i++) {
    if(cookies[i].getName().equals("nom")) {
        valeur=cookies[i].getValue();
    }
}
```

Voici l'ensemble des méthodes publiques de l'objet `Cookie` permettant d'obtenir des informations sur le cookie ou bien de préciser certains paramètres :

Méthode	Description
<code>Cookie(String Name, String Value)</code>	Constructeur créant un objet <code>Cookie</code> de nom <i>Name</i> et de valeur <i>Value</i>
<code>String getDomain()</code>	Retourne le domaine pour lequel le cookie est valide
<code>void setDomain(String Domain)</code>	Définit le domaine pour lequel le cookie est valide
<code>int getMaxAge()</code>	Retourne la durée de validité du cookie (en secondes)
<code>void setMaxAge(int duree)</code>	Définit la durée de validité du cookie (en secondes)
<code>String getName()</code>	Retourne le nom du cookie
<code>String getPath()</code>	Retourne le chemin pour lequel le cookie est valide

void setPath(String Chemin)	Définit le chemin pour lequel le cookie est valide
boolean getSecure()	Retourne <i>true</i> si le cookie doit être envoyé uniquement sur ligne sécurisée, sinon <i>false</i>
boolean setSecure(Boolean flag)	Définit si le cookie doit être envoyé sur une ligne sécurisée (<u>SSL</u>)
String getValue()	Retourne la valeur du cookie
void setValue(String Valeur)	Redéfinit la valeur du cookie
int getVersion()	Retourne la version du cookie
void setVersion(int Version)	Définit la valeur du cookie

Ce mécanisme est le plus intéressant pour stocker de petites informations. Toutefois l'utilisation de cookies pose quelques problèmes :

- des utilisateurs désactivent les cookies par crainte
- certains anciens navigateurs ne gèrent pas les cookies

5.2. Utilisation des objet de sessions

En premier lieu, il vous faut obtenir un objet de session. Pour ce faire, utilisez la méthode *getSession* fournie par l'objet de requête. Mais attention : deux cas peuvent se présenter à vous. Soit l'objet de session est déjà existant, et dans ce cas, il n'y aura aucun problème pour le récupérer. Soit il est inexistant. Dans cette dernière situation, le choix vous est laissé : soit vous souhaitez ne rien récupérer (valeur *null*). Soit vous préférez créer un nouvel objet de session. La méthode *getSession* accepte un paramètre booléen. Si vous passez la valeur *false*, et dans ce cas vous pouvez récupérer la valeur *null*. Sinon, vous passez *true* et un nouvel objet pourra être éventuellement créé.

La méthode *getSession* vous renvoie un objet de type *HttpSession*. L'extrait de code suivant vous montre comment récupérer, dans tous les cas, un objet de session.

```
// Obtention d'un objet de session
HttpSession session = request.getSession(true);
```

Une fois l'objet de session référencé, il vous faut pouvoir y stocker des informations et pouvoir les retrouver. Une session est en fait une table

associative : chaque données est associée à une clé (une chaîne de caractères). Pour y stocker une valeur, il faut préciser à quelle clé cette valeur est associée. Pour retrouver une information, il faut fournir la clé recherchée. Les lignes de code suivantes vous montre comment insérer des informations dans la session et comment les retrouver.

```
// Stockage dans l'objet de session
String login = request.getParameter("login");
session.setAttribute("login", login);

// Récupération dans l'objet de session
String login = (String)session.getAttribute("login");
```

Notez bien que chaque entrée de la session peut contenir un quelconque objet Java (y compris une chaîne de caractères). Lors de la récupération de la données, il est donc obligatoire de transtyper (cast) la valeur retournée en le type attendu (dans l'exemple précédent, *String*).

L'exemple suivant montre une servlet qui stocke, une fois le login et le password renseigné, le login de l'utilisateur dans la session. Cette information pourra ultérieurement être réutilisée par les autres servlets du site Web.

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class Login extends HttpServlet {
    public void doGet(HttpServletRequest request,
        HttpServletResponse response) throws ServletException,
        IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<HTML>");
        out.println("  <HEAD><TITLE>Login</TITLE></HEAD>");
        out.println("  <BODY>");
        out.println("    <DIV Align='center'>");
        out.println("      <H1>Veuillez vous authentifier !</H1>");
        out.println("      <FORM Method='POST'>");
```

```

        out.println("        Login : <INPUT Name='login'> <BR />");
        out.println("        Password : ");
        out.println("        <INPUT Name='password'
Type='password'>");
        out.println("        <BR /> <INPUT Type='submit'>");
        out.println("        </FORM>");
        out.println("        </DIV>");
        out.println("        </BODY>");
        out.println("</HTML>");
    }

    public void doPost(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        String login = request.getParameter("login");
        String password = request.getParameter("password");

        if (login.equals("") || password.equals("")) {
            response.sendRedirect("/login"); return;
        }

        HttpSession session = request.getSession(true);
        session.setAttribute("login", login);
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<HTML>");
        out.println("    <HEAD><TITLE>Login</TITLE></HEAD>");
        out.println("    <BODY>");
        out.println("        <DIV Align='center'>");
        out.println("            login + " - " + password + "<BR>");
        out.println("        </DIV>");
        out.println("    </BODY>"); out.println("</HTML>");
    }
}

```

Invalider une session

D'une manière générale, il est préférable de laisser une session se terminer seule (par expiration par exemple). Cependant, dans certains cas il peut être utile de supprimer manuellement une session). Pour supprimer une session, il suffit de faire appel à la méthode *invalidate()* de l'objet *HttpSession*

```

public class Delete_Session extends HttpServlet {
    public void doGet (HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException
    {
        // Recupere la session
        HttpSession session = request.getSession(true);
        session.invalidate();
    }
}

```

Autres méthodes de HttpSession

Méthode	Description
long getCreationTime()	Retourne l'heure de la création de la session
Object getAttributes(String Name)	Retourne l'objet stocké dans la session sous le nom <i>Name</i> , <i>null</i> s'il n'existe pas
String getId()	Génère un identifiant de session
long getCreationTime()	Retourne la date de la requête précédente pour cette session
String[] getValueNames()	Retourne un tableau contenant le nom de toutes les clés de la session
Object getValue(String Name)	Retourne l'objet stocké dans la session sous le nom <i>Name</i> , <i>null</i> s'il n'existe pas
void invalidate()	Supprime la session
boolean isNew()	Retourne <i>true</i> si la session vient d'être créée, sinon <i>false</i>

void putValue(String Name, Object Value)	Stocke l'objet <i>Value</i> dans la session sous le nom <i>Name</i>
void removeValue(String Name)	Supprime l'élément <i>Name</i> de la session
void setAttribute(String Name, Object Value)	Stocke l'objet <i>Value</i> dans la session sous le nom <i>Name</i>
int setMaxInactiveInterval(int interval)	Définit l'intervalle de temps maximum entre deux requête avant que la session n'expire
int getMaxInactiveInterval(int interval)	Retourne l'intervalle de temps maximum entre deux requête avant que la session n'expire

6. EXEMPLE 1: Accès à une base de données par des servlets

```
// package tomcat1;
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
import java.sql.*;

public class Gestion_DB extends HttpServlet {
    Connection con=null;
    Statement stmt=null;
    ResultSet rs = null;
    String url ="jdbc:odbc:essai";

    public void init (ServletConfig conf) throws ServletException {
        super.init(conf);

        try {
            Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
            con =DriverManager.getConnection(url,"","");
            stmt=con.createStatement();
        }

        catch (Exception e)
        {throw(new UnavailableException(this, "Sorry! The Database didn't load!"));
        }
    }
}
```

```
}
}

public void service (HttpServletRequest req, HttpServletResponse res)
throws ServletException, IOException {
    int nb=0 ;
    String requete;
    String url ="jdbc:odbc:essai";
    res.setContentType ( "text/html" );
    PrintWriter out = res.getWriter ( );
    String title = "JDBC Demo Java Servlet";
    out.println ( "<html><head><title>" + title
    + "</title></head>" );
    out.println ( "<body><H1>" + title + "</H1>" );
    String mcodecli = req.getParameter("Codecli");
    String mnom = req.getParameter("Nom");
    String madresse = req.getParameter("Adresse");
    String bt = req.getParameter ("BT");

    if (bt !=null)

        if (bt.equals("Ajouter"))
        {
            // AJOUTER
            try {
                requete ="INSERT INTO client(codecli,nom,adresse) VALUES('
                "+mcodecli + "',' ' +mnom + ' ', '"+madresse+"')";
                nb =stmt.executeUpdate(requete);
                out.println("<h3> ENREGISTREMENT AJOUTE </h3> ");
            }
            catch(Exception e) { out.println("<h3> ENREGISTREMENT DEJA
            EXISTANT </h3> ");
            }
        }

        else if (bt.equals("Supprimer"))
        {
            // SUPPRIMER
            try { nb =stmt.executeUpdate("DELETE FROM CLIENT where codecli =' "
            +mcodecli + "'");
            }
        }
    }
}
```

```

+ mcodecli + " ' ";
out.println("<h3> ENREGISTREMENT SUPPRIME </h3> ");
}
catch(Exception e) { out.println("ERROR DELETE");}
}

else if (bt.equals("Modifier"))
{ // MODIFIER
try {
nb =stmt.executeUpdate("UPDATE client set nom = ' " +mnom+ " ', " + "
adresse= ' " +madresse+" ' " + " WHERE codecli=' " +mcodecli+ " ' ");
out.println("<h3> ENREGISTREMENT MODIFIE </h3> ");
}
catch(Exception e) { out.println("ERROR UPDATE"); }
}

else if (bt.equals("Consulter"))
{ // CONSULTE
try {
rs =stmt.executeQuery("select * from client where codecli= ' " +mcodecli+" '
");
rs.next();
mnom= rs.getString("nom");
madresse =rs.getString("adresse");
out.println("<h3> CODECLI : " + mcodecli+"</h3> <BR>");
out.println("<h3> NOM : " + mnom+"</h3> <BR>");
out.println("<h3> ADRESSE : " + madresse+"</h3><BR>");
}

catch(Exception e){out.println("<h3> CODE CLIENT INCORRECT </h3> ");}
}

else

{

try
{

```

```

out.println("<table border>");
rs = stmt. executeQuery ("SELECT * FROM Client");
ResultSetMetaData rsMeta = rs. getMetaData();
// Get the N of Cols in the ResultSet
int noCols = rsMeta. getColumnCount ();
out.println("<tr>");
for (int c=1; c<=noCols; c++)
{
String elt = rsMeta. getColumnLabel (c);
out.println("<th> " + elt + " </th>");
}
out.println("</tr>");

while (rs.next())
{
out.println("<tr>");
for (int c=1; c<=noCols; c++)
{
String elt = rs. getString (c);
out.println("<td> " + elt + " </td>");
}
out.println("</tr>");
}
out.println("</table>");
}

catch(Exception ex) { out.println(" ERREUR LISTER");}

}

//out.println("< FORM
ACTION='http://localhost:8080/examples/servlet/tomcat1.Gestion_DB'>");
out.println("<FORM
ACTION='http://localhost:8080/examples/servlet/Gestion_DB'
METHOD='POST'>");

out.println("CodeCli :<INPUT TYPE=TEXT SIZE=50 NAME='Codecli'>

```

```

<BR>");
out.println("Nom   : <INPUT TYPE=TEXT SIZE=50 NAME='Nom'> <BR>");
out.println("Adresse : <INPUT TYPE=TEXT SIZE=50 NAME='Adresse'>
<BR>");

out.println("<INPUT TYPE=SUBMIT name='BT' VALUE='Ajouter'>");
out.println("<INPUT TYPE=SUBMIT name='BT' VALUE='Supprimer'>");
out.println("<INPUT TYPE=SUBMIT name='BT' VALUE='Modifier'>");
out.println("<INPUT TYPE=SUBMIT name='BT' VALUE='Consulter'>");
out.println("<INPUT TYPE=SUBMIT name='BT' VALUE='Lister'>");

out.println("</ FORM >");

out.println ( "</body></html>" );
return ;
// fin if bt
}
}

```

7. EXEMPLE 2 : une application Web comportant :

- une page JSP contenant une formulaire de saisie
- une servlet qui récupère les données du formulaire , calcule le montant du bonus en faisant appel à un Javabeau et renvoie le résultat à la page JSP

Bonus.jsp

```

<HTML>
<HEAD>
<TITLE>Bonus Calculation with MVC model</TITLE>
</HEAD>
<BODY BGCOLOR = "WHITE">
<BLOCKQUOTE>
<H3>Bonus Calculation with a JavaBean</H3>
<!--ACTION parameter calls this page-->

```

```

<FORM METHOD="GET" ACTION="
BonusCalculationServlet ">

```

```

<P>    Enter Social Security Number:
<P>    <INPUT TYPE="TEXT" NAME="ssn"></INPUT>

```

```

<P>    Enter Multiplier:
<P>    <INPUT TYPE="TEXT"
NAME="multiplier"></INPUT>

```

```

<P>    <INPUT TYPE="SUBMIT" VALUE="Submit">
<INPUT TYPE="RESET">
</FORM>

```

```

<!--Scriptlet and JavaBeans Tags start here -->
<jsp:useBean id="bonus"
class="bonus.beans.JBonusBean" scope="request"/>

```

```

Social Security Number retrieved:
<jsp:getProperty name="bonus" property="ssn"/>

```

```

<P>
Bonus Amount retrieved:
<jsp:getProperty name="bonus"
property="bonusAmount"/>

```

```

</BLOCKQUOTE>
</BODY>
</HTML>

```

JavaBeans Class : JBonusBean

```
package bonus.beans;
```

```
public class JBonusBean {
    private String ssn;

```

```

private double multiplier;
private double bonusAmount;

public JBonusBean() {}

public String getSsn(){
    return this.ssn;
}
public double getMultiplier(){
    return this.multiplier;
}
public double getBonusAmount() {
    return this.bonusAmount;
}
public void setSsn(String newSsn) {
    this.ssn = newSsn;
}
public void setMultiplier(double newMultiplier) {
    this.multiplier = newMultiplier;
}
public void setBonusAmount(double newBonusAmount) {
    this.bonusAmount = newBonusAmount;
}
public void calculateBonus(double bonus) {
    this.bonusAmount = (this.multiplier * bonus);
}
}

```

Servlet : **BonusCalculationServlet**

```

package bonus.controllers;
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
import java.util.*;
import bonus.beans.*;

```

```

public class BonusCalculationServlet extends HttpServlet {
    public void init(ServletConfig config) throws
        ServletException {
        System.out.println("BonusCalculationServlet: init()");
    }

    public void doGet (HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {
        System.out.println("BonusCalculationServlet: doGet()");
        // create the JavaBean model
        JBonusBean bonusBean= new JBonusBean();
        // Retrieve the SSN Information
        String ssn = request.getParameter("ssn");
        // set the ssn property in JBonusBean
        bonusBean.setSsn(ssn);
        // Retrieve Bonus Multiplier information
        String multiplierAsString =
        request.getParameter("multiplier");
        int multiplier = Integer.parseInt(multiplierAsString);
        // set the multiplier property in the JBonusBean
        bonusBean.setMultiplier(multiplier);
        //Calculate bonus
        double bonus = 100.00;
        bonusBean.calculateBonus(bonus);
        request.setAttribute("bonus", bonusBean);
        request.getRequestDispatcher("bonus.jsp").forward(re
quest, response);
    }

    public void destroy() {
        System.out.println("BonusCalculationServlet: destroy()");
    }
}

```

8. Exemple 3 : Accès à une base de données PostGresql

```
/* A servlet to display the contents of the PostgreSQL Bedrock database */
```

```
import java.io.*;
import java.sql.*;
import java.text.*;
import java.util.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class ShowBedrock extends HttpServlet
{
    public String getServletInfo()
    {
        return "Servlet connects to PostgreSQL database and displays
result of a SELECT";
    }

    private Connection dbcon; // Connection for scope of ShowBedrock
    // "init" sets up a database connection
    public void init(ServletConfig config) throws ServletException
    {
        String loginUser = "postgres";
        String loginPasswd = "supersecret";
        String loginUrl = "jdbc:postgresql://localhost/bedrock";

        // Load the PostgreSQL driver
        try
        {
            Class.forName("org.postgresql.Driver");
            dbcon = DriverManager.getConnection(loginUrl, loginUser,
loginPasswd);
        }
        catch (ClassNotFoundException ex)
        {
            System.err.println("ClassNotFoundException: " +
ex.getMessage());
            throw new ServletException("Class not found Error");
        }
        catch (SQLException ex)
        {

```

```
        System.err.println("SQLException: " + ex.getMessage());
    }

    // Use http GET

    public void doGet(HttpServletRequest request, HttpServletResponse
response)
        throws IOException, ServletException
    {
        response.setContentType("text/html"); // Response mime type

        // Output stream to STDOUT
        PrintWriter out = response.getWriter();

        out.println("<HTML><Head><Title>Bedrock</Title></Head>");
        out.println("<Body><H1>Bedrock</H1>");

        try
        {
            // Declare our statement
            Statement statement = dbcon.createStatement();
            String query = "SELECT name, dept, ";
            query += "      jobtitle ";
            query += "FROM   employee ";
            // Perform the query
            ResultSet rs = statement.executeQuery(query);
            out.println("<table border>");
            // Iterate through each row of rs
            while (rs.next())
            {
                String m_name = rs.getString("name");
                String m_dept = rs.getString("dept");
                String m_jobtitle = rs.getString("jobtitle");
                out.println("<tr>" +
                    "<td>" + m_name + "</td>" +
                    "<td>" + m_dept + "</td>" +
                    "<td>" + m_jobtitle + "</td>" +
                    "</tr>");
            }

```

```

        out.println("</table></body></html>");
        statement.close();
    }
    catch(Exception ex)
    {
        out.println("<HTML>" +
            "<Head><Title>" +
            "Bedrock: Error" +
            "</Title></Head>\n<Body>" +
            "<P>SQL error in doGet: " +
            ex.getMessage() + "</P></Body></HTML>");

        return;
    }
    out.close();
}
}

```

9. La mise en oeuvre des servlets

L'installation Jakarta tomcat sur Windows 9x

Il est possible de récupérer tomcat sur le site [de Jakarta](#). Il faut choisir la version stable de préférence et le répertoire bin pour récupérer le fichier jakarta-tomcat.zip

Dézipper le fichier par exemple dans C:\. L'archive est décompressée dans un répertoire nommé jakarta-tomcat

Dans une boîte DOS, assigner le répertoire contenant tomcat dans une variable d'environnement TOMCAT_HOME

Exemple : set TOMCAT_HOME=c:\jakarta-tomcat

Lancer Tomcat en executant le fichier startup.bat dans le répertoire TOMCAT_HOME\bin

Vérifier que Tomcat s'exécute en saisissant l'url <http://localhost:8080> dans un browser. La page d'accueil de Tomcat s'affiche.

Le script %TOMCAT_HOME%\bin\shutdown.bat permet de stopper Tomcat.

Compilation d'une servlet

On peut compiler une servlet par la commande :

```

javac -d [directory] -classpath repertoire/servlet.jar
fichier_servlet.java

```

Exemple 1 : Simple Ajax script for JSP

index.jsp

```

<html><head><script src="ajaxjs.js"></script>

</head>

<body>

<a href="javascript: loadContent('parameterValue')">Load Ajax content</a>

<div id="prtCnt"></div>

</body>

</html>

```

loadJSP.jsp

```

<%@ page contentType="text/html; charset=iso-8859-1" language="java"
%>

<% String q =request.getParameter("q");

String str="This is JSP string loading from JSP page in ajax, loading time :";

java.util.Date dt=new java.util.Date();

out.print(str+dt);

%>

```

ENI
ajaxjs.js

```
var xmlhttp

function loadContent(str)
{ xmlhttp=GetXmlHttpRequest();
  var url="loadJSP.jsp";
    url=url+"?q="+str;
    xmlhttp.onreadystatechange=getOutput;
    xmlhttp.open("GET",url,true);
    xmlhttp.send(null);
}

function getOutput(){
if (xmlhttp.readyState==4)
{ document.getElementById("prtCnt").innerHTML=
  xmlhttp.responseText;
}
}
```

Exemple 2: JSP+ SERVLET +AJAX

Index.html

RV 23

```
<%@ page language="java" contentType= "text/html; charset=ISO-8859-1"%>
```

```
<html><head>
```

```
<script src = "script.js" type="text/javascript">
```

```
</script>
```

```
</head>
```

```
<body>
```

```
<table>
```

```
<tr><td>Valeur :</td>
```

```
<td nowrap><input type="text" id="donnees" name="donnees" size="30"
onkeyup="valider();"></td>
```

```
<td><div id="validationMessage"></div></td>
```

```
</tr>
```

```
</table>
```

```
</body>
```

```
</html>
```

WEB.XML

```
<servlet-mapping>
```

```
<servlet-name>
```

```
ValiderServlet
```

```
</servlet-name>
```

```
<url-pattern>/
```

ENI

validerServlet

</url-pattern>

</servlet-mapping>

script.js

var requete;

function valider() {

var donnees = document.getElementById("donnees");

var url = "ValiderServlet?valeur=" + escape(donnees.value);

var requete=getRequeteHttp();

requete.open("GET", url, true);

requete.onreadystatechange = majIHM;

requete.send(null);

}

function majIHM() {

var message = "";

if (requete.readyState == 4) {

if (requete.status == 200) {

// exploitation des données de la réponse

var messageTag
requete.responseXML.getElementsByTagName("message")[0];

RV 24

message = messageTag.childNodes[0].nodeValue;

mdiv = document.getElementById

("validationMessage");

if (message == "invalide") {

mdiv.innerHTML = "";

} else {

mdiv.innerHTML = "";

}

} }}

ValiderServlet.java

import java.io.IOException;

import javax.servlet.*;

import javax.servlet.http.* ;

public class ValiderServlet extends HttpServlet {

protected void doGet(HttpServletRequest request, HttpServletResponse
response)

throws ServletException, IOException {

String resultat = "invalide";

String valeur = request.getParameter("valeur");

response.setContentType("text/xml");

if ((valeur != null) && valeur.startsWith("X")) {

resultat = "valide";

} response.getWriter().write("<message>" + resultat + "</message>"); }

}

=